Guida al sistema

Come è fatta la casa editrice del repository ZeusBot: chi fa cosa, il percorso di un libro, le regole.

Per Felice, autore. Ottobre 2026.

Fotografia del ramo `integrazione-editoriale` al commit `951a42c`.

> **Come leggere.** Tra parentesi, in grigio, c'è il file da cui viene ogni affermazione. Quando una cosa non c'è nel repository, la guida lo scrive: «non esiste».

> **Questa edizione** descrive il ramo `integrazione-editoriale` (commit `951a42c`): la base del ramo di sessione ccr con le aggiunte del ramo `motore`. Le scelte dell'unione sono in

> `docs/DECISIONI_INTEGRAZIONE.md` (D1-D18) (docs/DECISIONI_INTEGRAZIONE.md, introduzione).

Indice

0. Le parole da sapere 4
1. A cosa serve il sistema 8
2. Organigramma: chi fa cosa 10
3. Il percorso di un libro 30
4. Le regole 52
5. I rami 61
6. Cosa c'è e cosa non esiste 64
7. Appendici e indice 68
8. Punti deboli e decisioni aperte 70
9. Dove trovare le cose 75

0. Le parole da sapere

Poche parole tecniche tornano spesso. Eccole, una volta per tutte.

- **Repository**: l'archivio del progetto su GitHub, con tutta la storia delle modifiche. Qui si chiama ZeusBot.
- **Ramo**: una versione parallela del repository. Ogni ramo ha i suoi file. Il ramo `motore` contiene la macchina; ogni libro vive su un ramo suo.
- **Commit**: un salvataggio con nome e data. Ha un codice breve, per esempio `078b9b3`.
- **Push**: l'invio dei salvataggi su GitHub (su «origin», la copia in rete). Finché non c'è il push, il lavoro esiste solo nel computer di lavoro.
- **Merge**: l'unione di un ramo dentro un altro.
- **Script**: un piccolo programma che fa un lavoro preciso (contare parole, fare il PDF…).
- **Hook**: un guardiano automatico. Claude Code lo chiama prima o dopo

certe azioni; l'hook può lasciar passare, avvisare o bloccare.

- **Claude Code**: l'assistente che lavora nel repository, legge le regole e lancia i comandi.
- **YAML**: un formato di testo per i dati, fatto di righe «nome: valore». I file finiscono in `.yaml`.
- **Gate** (fermata): il lavoro si ferma finché l'autore non risponde. Nei libri nuovi resta una sola fermata dell'autore, la pausa di lettura.
- **Modo automatico**: la scrittura va avanti da sola, senza approvazioni intermedie; i controlli li fa il motore. È una decisione dell'autore del 20 ottobre 2026.
- **Pausa di lettura**: ogni 3 capitoli (da 3 a 5, si sceglie in `libro.yaml`) il motore si ferma e ti dà l'elenco dei capitoli da leggere.
- **Struttura**: la prima fase dopo il briefing. Prima della prosa si fissano premessa, personaggi, svolte, scaletta, ritmo e scelte fisse (punto di vista, tempo, voce).

- **Cartella unica**: tutto ciò che riguarda un libro vive in `libri/<slug>/`, e il nome del libro non compare altrove.
- **Ruolo**: un mestiere editoriale (diagnosta, direttore, architetto, scrittore, editor, correttori). Una sessione di Claude Code svolge un ruolo solo.
- **Subagente** (o agente): un ruolo che Claude Code chiama come aiutante, con un contesto suo e pulito. Risponde con al massimo 10 righe.
- **Capsula**: il riassunto di continuità di un capitolo, in `07-capsule/NN.md`. Chi scrive legge le capsule, non i capitoli interi.
- **Audit anti-IA**: il controllo che segnala i segni di scrittura da IA. Il motore segnala; non riscrive.
- **sha256** (impronta): l'«impronta digitale» di un file. Se cambia anche una virgola, l'impronta cambia.
- **KDP**: Kindle Direct Publishing, il servizio di Amazon per pubblicare.
- **Metadati del PDF**: dati nascosti nel file, come Title (titolo) e Author

(autore).

- **Slug**: il nome breve di un libro usato nei percorsi, senza spazi né accenti.

(motore/editoriale/glossario.md; docs/DECISIONI_INTEGRAZIONE.md, D4 e D11; motore/script/fase.py)

1. A cosa serve il sistema

Il sistema è una casa editrice per romanzi di qualsiasi genere (motore/README.md; motore/editoriale/LEGGIMI.md).

Il cuore è il **motore**: procedura, script, modelli vuoti, profili di genere, dati KDP e il livello editoriale (percorso, ruoli, agenti, anti-IA, correttore). Il motore non contiene libri veri (motore/README.md; motore/editoriale/LEGGIMI.md).

Il percorso è sempre lo stesso. L'autore scrive un briefing. Il motore crea la cartella del libro. Prima la struttura, poi i documenti, poi i capitoli, uno alla volta. Seguono le revisioni, la chiusura meccanica, il PDF di stampa e il pacchetto per KDP (motore/editoriale/00-percorso.md).

Cosa cambia con il modo automatico. Nei libri nuovi il motore scrive senza chiedere approvazioni a ogni tappa. Si ferma per te solo alla **pausa di lettura**,

ogni 3 capitoli (docs/DECISIONI_INTEGRAZIONE.md, D4, D6, D11).

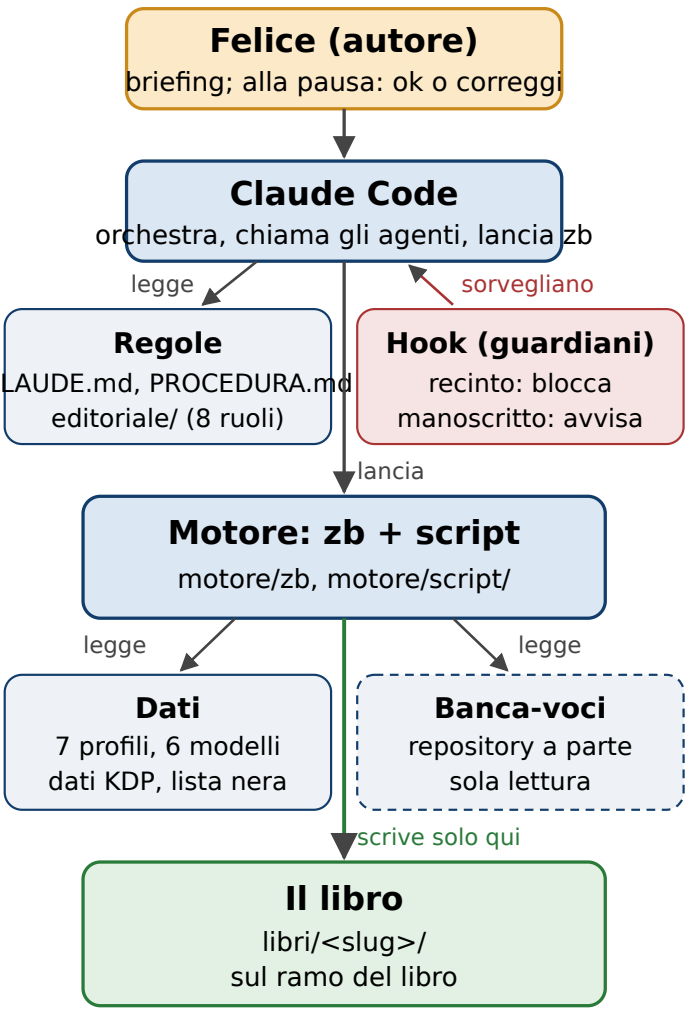
Tre idee reggono tutto (CLAUDE.md):

1. **Il motore resta pulito.** Il ramo `motore` non contiene mai un libro.
2. **Si scrive solo nella cartella del libro.** Mai nel motore, mai altrove.
3. **Proponi, l'autore approva, poi applica.** Claude Code non approva mai da solo.

> La terza regola e il modo automatico vanno letti insieme: nei libri nuovi l'unica approvazione durante la scrittura è la tua risposta alla pausa di lettura. I file del livello editoriale descrivono ancora le fermate di un tempo: il contrasto è segnalato in `docs/DA_VERIFICARE_AUTORE.md`. `CLAUDE.md` invece dice tutte e due le cose, la regola e il modo automatico (docs/DECISIONI_INTEGRAZIONE.md, D4 e D17).

2. Organigramma: chi fa cosa

Lo schema



Schema ricavato da CLAUDE.md, motore/editoriale/LEGGIMI.md, motore/dati/hook.yaml e motore/dati/hook-settings.esempio.json.

Chi fa cosa, in breve

Chi	Che cosa fa	Dove sta
Felice	Scrive il briefing; alla pausa di lettura risponde ok o correggi; decide	—
Sessione principale (Claude Code)	Orchestra: chiama l'agente del passo, legge la sua risposta breve, lancia `zb`. **Non scrive prosa**	—

Chi	Che cosa fa	Dove sta
Ruoli e agenti	Fanno il lavoro editoriale	`motore/editoriale/`
Motore	Crea, controlla, compone, impagina	`motore/`
Regole	Dicono come si lavora	`CLAUDE.md`, `motore/PROCEDURA.md`, `motore/editoriale/`
Hook	Sorvegliano le scritture	`motore/script/hook_*.py`, `consumo.py`
Libro	Il lavoro vero	`libri/<slug>/`

(CLAUDE.md; motore/editoriale/LEGGIMI.md; motore/editoriale/risparmio-token.md)

Gli otto ruoli editoriali

Ogni ruolo ha un file in `motore/editoriale/ruoli/`. Legge solo i file del suo «Input» e scrive un report in `06-diagnostica/` del libro. Non modifica il manoscritto, salvo chi scrive e chi corregge (motore/editoriale/LEGGIMI.md, «Regole»).

#	Ruolo	Lavoro	Fase
0	Diagnosta	dice cosa fa il testo e dove non funziona, senza riscrivere: `rapporto-struttura.md`, `rapporto-manoscritto.md`; in stesura `audit/NN.md`, un capitolo alla volta	`struttura`, `stesura` (solo con N), `struttura_manoscritto`

#	Ruolo	Lavoro	Fase
1	Direttore editoriale	bibbia, schede del codex, cronologia, nota di direzione	`documenti`
2	Architetto della struttura	i nove file di `03-struttura/`	`struttura`
3	Scrittore con voce	pagina campione e capitoli	`pagina_campione`, `stesura`

#	Ruolo	Lavoro	Fase
4	Editor strutturale	trasforma la diagnosi in un piano di revisione: `piano-revisione-struttura.md`, `piano-revisione-manoscritto.md`	`struttura`, `struttura_manoscritto`
5	Line editor	frase per frase: ritmo, voce, ripetizioni, tic da IA	`line`

#	Ruolo	Lavoro	Fase
6	Correttore di italiano	grammatica, ortografia, punteggiatura, coerenza	`copy`
7	Correttore di bozze	ultimo controllo sul libro impaginato	`bozze`

(motore/editoriale/ruoli/00-07; motore/script/percorso.py, FASI_RUOLO)

`zb ruolo <libro> <ruolo> [N]` rifiuta un ruolo fuori dalla sua fase e dà la prima riga del rapporto, con l'impronta del testo rivisto (motore/script/ruolo.py).

Gli agenti

Gli agenti sono 12. In `motore/editoriale/agenti/` ce ne sono 10: uno per ruolo, più il **lettore cieco**

(passo facoltativo) e la **capsula**. In `motore/editoriale/lingue/agenti/` ci sono i 2 della traduzione: traduttore e revisore di fedeltà (cartelle motore/editoriale/agenti/ e lingue/agenti/; zb agenti stato).

- **Modelli.** Forte (opus): architetto, editor strutturale. Medio (sonnet): direttore, scrittore, line editor, correttore di italiano, correttore di bozze, lettore cieco, traduttore, revisore di fedeltà. Leggero (haiku): capsula e diagnosta (dal 2026-10-21, il più economico tra gli alias ammessi; da provare su una pagina: se la revisione è debole, si torna al medio). Fonte unica: `motore/dati/modelli-ruoli.yaml` (motore/dati/modelli-ruoli.yaml).
- **In sequenza:** scrittore e traduttore, un capitolo per volta. **In parallelo:** line editor, correttore di italiano e capsula, al massimo 3 alla volta; con i limiti d'uso si scende a 2, poi a 1 (motore/dati/orchestrazione.yaml; motore/dati/modelli-ruoli.yaml).

- **Chi chiamare ora:** `zb agenti prossimo <libro>`. Non scrive nulla (motore/script/agenti.py).
- **Risposta breve:** ogni agente risponde con al massimo 10 righe e non fa commit né push (motore/dati/orchestrazione.yaml; motore/editoriale/agenti/).
- **Il lettore cieco** legge un campione da lettore, in sola lettura, solo dopo il tuo ok (motore/editoriale/agenti/lettore-cieco.md; motore/dati/orchestrazione.yaml).
- **Comandi di Claude Code:** `/scrivi-capitolo`, `/revisiona`, `/diagnosi`, `/handover` (motore/editoriale/comandi/).
- **Accensione:** `zb agenti attiva --ok` copia agenti e comandi in `.claude/agents/` e `.claude/commands/`: 16 file, solo sui rami dei libri (motore/script/agenti.py; zb agenti stato).

Le capsule

Dopo ogni capitolo accettato: `zb capsula <libro> N` prepara `07-capsule/NN.md`, l'agente capsula la scrive, `zb capsula <libro> N --verifica` la controlla. Otto sezioni, da 120 a 250 parole (circa 180). Le promesse si scrivono con `@prom:<id>`; il registro segnala una promessa chiusa prima di essere aperta (motore/script/capsula.py).

`zb esito` si ferma se manca la capsula del capitolo precedente. Con `capsule: obbligatorie` in `libro.yaml` si ferma anche `zb avvio`; senza il campo dà un avviso (motore/PROCEDURA.md §1b; motore/script/capsula.py; docs/DECISIONI_INTEGRAZIONE.md, D9).

Il motore: `zb` e gli script

`zb` è il comando unico. Si scrive `python3 motore/zb <comando> <libro>`. Il libro si indica sempre con il percorso della sua cartella. L'elenco dei comandi è uno solo, `motore/dati/comandi.yaml`: lo stampa `python3 motore/zb aiuto`, e `zb aiuto`

`<comando>` dà l'uso (motore/PROCEDURA.md §11; motore/dati/comandi.yaml).

Gli script sono 44 file in `motore/script/`. I comandi sono 47, in quattro gruppi (cartella motore/script/; motore/dati/comandi.yaml):

- **Libro e fasi:** nuovo, adotta, avvio, stato, pronto, avanti, ok, correggi, esito, riapprova, fase, canone, revisione.
- **Percorso editoriale e agenti:** struttura, codex, prepara, capsula, compatto, ruolo, agenti, consumo, antiia, traduci.
- **Controlli e chiusura:** capitolo, conta, stile, continuita, riciclo, ortografia, languagetool, compila, impagina, pdf, pacchetto, kdp, verifica.
- **Recinto, salvataggio, motore:** lavoro, salva, push, cartella, hook, pulizia, voce, prove, separazione, recinto, profili.

Ogni comando stampa al massimo 40 righe e mette il resto in un file; le righe con KO, FERMO o ERRORE restano sempre a schermo (motore/dati/output.yaml).

Nessun comando del motore fa commit o push da solo, tranne `zb salva` e `zb push` quando li si lancia (motore/PROCEDURA.md §8).

La «Bibbia editor»

«Bibbia editor» era il nome dato all'insieme delle regole editoriali: il commit `668afc2` si chiama «Bibbia editor: appendici e indice» (storia git). Oggi quelle regole stanno in due posti:

- `motore/editoriale/`: percorso, ruoli, diagnosi, agenti, struttura, anti-IA, correttore, glossari;
- `motore/PROCEDURA.md`: la procedura completa.

Le aiutano alcuni file di dati, per esempio la lista nera di base (`motore/dati/lista-nera.yaml`) e le parole italiane ammesse

(`motore/dati/parole-ammesse-it.txt`) (motore/editoriale/LEGGIMI.md; motore/PROCEDURA.md §7).

> Attenzione: la `bibbia.md` di ogni libro (personaggi, luoghi, fatti) è un'altra cosa. Sta nella cartella del libro, in `02-bibbia/` (motore/dati/struttura-libro.yaml).

Profili di genere

Un profilo dà i valori predefiniti di un genere: lunghezza, capitoli, scene, frase media, dialogo e soglie anti-IA (motore/PROCEDURA.md §3; motore/profili/).

Sono 7, nella cartella `motore/profili/`: fantascienza, fantasy, giallo, narrativa letteraria, romance, thriller, young adult. Le soglie anti-IA dei profili sono una PROPOSTA, oggi uguale per tutti i generi (motore/profili/giallo.yaml, blocco anti_ia).

Un valore del briefing fuori dal profilo è accettato solo con un «Motivo override:»

scritto nel briefing. Senza motivo, `zb nuovo` si ferma (motore/PROCEDURA.md §3).

Nel thriller la frase media la dà la voce: con `frase_media_voce: true` ogni capitolo si misura sull'intervallo del suo registro nella scheda della voce (corto, freddo, base), dichiarato nella scaletta. Fuori intervallo è un AVVISO, non un blocco: l'intervallo è una scelta della voce, non ancora calibrata sui tuoi testi. Un override in `libro.yaml` resta sopra (motore/profili/thriller.yaml; motore/PROCEDURA.md §6).

Voci narranti

Una voce dice **come** si racconta. Il **cosa** lo decide l'autore nel briefing (motore/README.md, «Banca-voci»).

Le voci stanno in un repository separato, la banca-voci, mai dentro il motore. Il motore le legge in sola lettura dalla cartella indicata da `ZB_BANCA_VOCI`. I nomi delle voci stanno solo lì: questa guida non

li scrive (motore/passaggio-di-consegne.md, «Stato»).

Ogni voce ha una scheda in 7 sezioni e un elenco di espressioni vietate, `divieti.yaml`. `zb voce lista`, `zb voce mostra <slug>` e `zb voce verifica <slug>` le leggono (motore/README.md, «Banca-voci»). Un divieto può avere `massimo: N`: l'avviso scatta solo oltre N volte nel capitolo; più divieti nello stesso `gruppo` contano insieme contro un solo massimo (per esempio tutte le forme di chiusa). La sezione 6 della scheda può avere esempi «versione debole → versione forte»; `zb prepara` li nomina allo scrittore solo se ci sono, e si ferma se la scheda della voce manca (motore/README.md; motore/script/prepara.py).

Un libro sceglie la voce nel briefing («Voce dalla banca:», «Ambientazione:»); `zb nuovo` la scrive in `libro.yaml`. Se il briefing contrasta con la voce, vince il briefing, e il contrasto si segnala

(motore/README.md, «Banca-voci»; motore/editoriale/LEGGIMI.md).

Modelli

I modelli sono i file vuoti da cui nasce un libro. Stanno in `motore/modelli/` e sono 6 (cartella motore/modelli/):

File	A cosa serve
`briefing.md`	Il modulo che compila l'autore
`libro.yaml`	Le regole del libro
`stato.yaml`	A che punto è il libro
`cronologia.yaml`	Date ed eventi
`LEGGIMI.md`	Cosa leggere a ogni avvio
`conferme-autore.yaml`	Le conferme per KDP

I modelli dei documenti della struttura stanno a parte, in `motore/editoriale/struttura/modelli/` (cartella

motore/editoriale/struttura/modelli/; docs/DECISIONI_INTEGRAZIONE.md, D6).

Hook

Gli hook sono quattro, nel modello `motore/dati/hook-settings.esempio.json`. Gli effetti, uno per uno, sono nel capitolo 4.

Hook	Quando	Cosa fa
sessione	all'inizio della sessione	Annota la sessione; non blocca mai
manoscritto	prima di scrivere in `04-manoscritto/`	Oggi **avvisa** se manca `zb avvio`

Hook	Quando	Cosa fa
recinto	prima di ogni scrittura, anche via Bash, e prima di `git commit` e `git push`	**Blocca**
consumo	dopo le letture e le chiamate agli agenti	Annota, non blocca mai

(motore/dati/hook-settings.esempio.json; motore/dati/hook.yaml; motore/script/hook_*.py; motore/script/consumo.py)

Si accendono solo sui rami dei libri, con `zb hook attiva --ok`. Sul ramo `motore` sono spenti per scelta (motore/PROCEDURA.md §10).

CLAUDE.md

È il foglio delle regole di avvio. Sta nella radice del repository e Claude Code lo legge a ogni sessione. Ha al massimo 30 righe e nessun nome di libro: lo controlla una prova del motore (motore/script/prove.py, sezione CLAUDE.md). Cosa dice: capitolo 4.

Rami dei libri

Ogni libro vive su un ramo suo, nella cartella `libri/<slug>/`. Dettagli nel capitolo 5.

3. Il percorso di un libro

Dall'idea a KDP il percorso ha dodici passi. La tabella dice dove si ferma un libro **nuovo** del percorso editoriale, nel modo automatico (motore/editoriale/00-percorso.md; motore/script/nuovo.py; motore/script/percorso.py).

#	Passo	Chi lavora	Fermata dell'autore
1	Briefing	autore	—
2	Cartella unica `libri/<slug>/`	`zb nuovo`	—
3	Struttura: nove file	architetto della struttura	—
4	Revisione della struttura	diagnosta, poi editor strutturale	— (la controlla il motore)

#	Passo	Chi lavora	Fermata dell'autore
5	Documenti: bibbia, codex, cronologia, manuale	direttore editoriale	—
6	Pagina campione	scrittore con voce	—
7	Stesura, un capitolo alla volta	scrittore, poi capsula	**pausa di lettura** ogni 3 capitoli e a fine stesura
8	Revisione strutturale del manoscritto	diagnosta, poi editor strutturale	—

#	Passo	Chi lavora	Fermata dell'autore
9	Line editing	line editor	—
10	Copyediting	correttore di italiano	—
11	Bozze	correttore di bozze	—
12	Chiusura meccanica	motore	—

Dopo il passo 8 c'è un **passo facoltativo**: il lettore cieco, solo con il tuo ok (motore/dati/orchestrazione.yaml).

Il modo automatico

- I libri nuovi del percorso editoriale nascono con `gate: [lettura]` e `pausa_lettura: 3` in `libro.yaml` (motore/script/nuovo.py).

- La struttura la verifica il motore, sempre, senza la tua fermata: anche se `libro.yaml` elenca il gate `struttura` (motore/script/fase.py, gate_attivi; motore/script/percorso.py).
- Nelle fasi di revisione (passi 8-11) un gate non elencato non ferma: le proposte si registrano e si passa all'applicazione (motore/script/percorso.py, pronto).
- Restano due fermate che non sono approvazioni del lavoro: `controllo_fallito` (stesso controllo fallito due volte sullo stesso capitolo) e `canone` (cambiare i fatti bloccati è una tua richiesta) (docs/DECISIONI_INTEGRAZIONE.md, D6).
- Un briefing può ancora chiedere dei gate, con la riga «Gate:». I libri nati prima e quelli del percorso breve non cambiano (motore/script/nuovo.py; docs/DECISIONI_INTEGRAZIONE.md, D6 e D11).

La pausa di lettura: cosa rispondi

Dopo `pausa_lettura` capitoli accettati da `zb esito`, e a fine stesura, il motore si ferma. Stampa «PAUSA DI LETTURA» con l'elenco dei file da leggere. Finché non rispondi, `zb esito` rifiuta altri capitoli (motore/script/fase.py, apri_pausa e cmd_esito).

Leggi i capitoli nei file. Poi rispondi in uno di due modi (motore/script/fase.py, chiudi_pausa, cmd_ok, cmd_correggi):

- **ok**: Claude Code lancia `zb ok <libro>`. I capitoli sono registrati come letti, con impronta e commit, e la scrittura riparte.
- **correggi** con le tue note: Claude Code lancia `zb correggi <libro> <note>`. Le note vanno in `06-diagnostica/note-lettura.md` e in `stato.yaml` (`note_lettura`), poi la scrittura riparte. Le note valgono per i capitoli successivi: `zb prepara` le mette tra i file che lo scrittore legge.

Altre cose da sapere:

- Non servono blocchi da mostrare in chat: l'«ok» non chiede di aver visto i blocchi, perché leggi i file. Se li vuoi comunque, `zb avanti` li mostra (docs/DECISIONI_INTEGRAZIONE.md, D11).
- Un capitolo già scritto si corregge a parte, con `zb correggi <libro> <unità>: <motivo>` (docs/DECISIONI_INTEGRAZIONE.md, D11; motore/PROCEDURA.md §5).
- Valori ammessi per `pausa_lettura`: 3, 4 o 5. Con un altro valore ogni comando si ferma e lo dice (motore/dati/libro.schema.yaml).

Prima di ogni sessione: `zb avvio`

A ogni sessione, e dopo ogni compattazione della memoria di Claude Code, si lancia `zb avvio <libro> [--ruolo R]`. Si ferma senza scrivere nulla se qualcosa non va: modifiche non salvate, ramo sbagliato, ramo non allineato a GitHub, un documento approvato

cambiato di nascosto, file cambiati fuori dal libro, un altro libro attivo, un secondo ruolo nella stessa sessione (motore/script/avvio.py; motore/PROCEDURA.md §4).

Se tutto va bene stampa la riga «Letto: …», con i file letti, le righe e le impronte; il file del ruolo entra nella riga. Claude Code la riporta prima di scrivere. Nelle sessioni nuove si legge lo stato compatto del libro (15-20 righe) e la scheda del capitolo (motore/script/avvio.py; CLAUDE.md).

Un ruolo per sessione. Senza `--ruolo` vale quello della fase. Per cambiare ruolo: salvare, aprire una sessione nuova, `zb avvio` (motore/script/ruolo.py; motore/editoriale/00-percorso.md).

Passi 1-2. La nascita

1. Sul ramo del libro, mai sul ramo `motore`, l'autore compila il modello `motore/modelli/briefing.md`.
2. Claude Code lancia `zb nuovo <briefing>`.

3. Se il briefing è incompleto, `zb nuovo` elenca al massimo 5 cose mancanti e **non crea nulla**.
4. Se è completo, crea la cartella `libri/<slug>/` con `libro.yaml`, `stato.yaml`, `LEGGIMI.md`, `nomi_propri.txt`, e in `03-struttura/` i nove modelli della struttura. Bibbia, codex, cronologia e manuale nascono dopo, quando il motore chiude la struttura.

Per un libro che ha già dei documenti c'è `zb adotta`: prepara delle bozze con la fonte di ogni valore (motore/script/nuovo.py; motore/PROCEDURA.md §3 e §3b).

Passi 3-4. Struttura e sua verifica

L'architetto scrive nove file in `03-struttura/`: premessa, logline e tema, personaggi chiave, atti e svolte, scaletta, sottotrame e temi, tensione e ritmo, piano parole, scelte fisse

(motore/editoriale/struttura/01-fase-struttura.md).

Poi, in sessioni separate: il diagnosta scrive `06-diagnostica/rapporto-struttura.md`; l'editor strutturale scrive `06-diagnostica/piano-revisione-struttura.md`. I rapporti stanno in commit che non toccano `03-struttura/` (motore/editoriale/struttura/01-fase-struttura.md; motore/editoriale/00-percorso.md).

`zb pronto` verifica tutto: i nove file senza la riga di modello, macrostruttura dichiarata e motivata, schede complete, rapporti con l'impronta della struttura attuale. Se regge, registra i file, crea i documenti e passa a «documenti», senza fermata. Se no, si ferma e scrive il motivo (motore/script/percorso.py, pronto; motore/PROCEDURA.md §1b).

Passi 5-6. Documenti e pagina campione

Fase	Lavoro	Controllo di `zb pronto`
documenti	bibbia, schede del codex, cronologia, manuale; `06-diagnostica/nota-direzione.md`	`zb codex verifica`; senza la nota di direzione si ferma
pagina campione	`05-revisioni/pagina-campione.md`	pagina non vuota

(motore/PROCEDURA.md §1b; motore/script/percorso.py, documenti_gate; docs/DECISIONI_INTEGRAZIONE.md, D12)

Passo 7. La stesura

Per ogni capitolo, la sessione principale chiama l'agente scrittore, mai due capitoli in parallelo (motore/editoriale/agenti/scrittore-voce.md; motore/PROCEDURA.md §1b e §6):

1. `zb prepara <libro> N`: scheda e parole previste del capitolo (lo scrittore non rilegge la scaletta e il piano parole interi), schede del codex della scena, canone, capsule, le sezioni da leggere della voce e, dopo una pausa di lettura con note, le tue note;
2. lo scrittore scrive `04-manoscritto/NN-<slug>.md`;
3. revisione del capitolo: prima le misure del motore. `zb ruolo <libro> diagnosta N` controlla il capitolo (frase media, dialogo, divieti, ripetizioni, gli altri controlli di `zb esito`): con un KO o un avviso dei divieti della voce si ferma e il diagnosta non si chiama; frase media e ripetizioni restano avvisi (soglie ancora da confermare) e

passano al diagnosta tra le righe misurate. Con le misure superate scrive `06-diagnostica/misure/NN.md` (misure, righe misurate, file da leggere, canone) e le frasi finali; la sessione principale passa al diagnosta la prima riga e quel file (l'agente non ha la shell). Il diagnosta, in un'altra sessione, scrive `06-diagnostica/audit/NN.md` con «## Da correggere» e chiude con «passa» o «non passa». Un solo giro completo: il secondo rivede solo le frasi segnalate dal primo; dopo il secondo «non passa» decidi tu. Con `revisione_capitolo: obbligatoria` in `libro.yaml`, `zb esito` non accetta il capitolo senza un «passa» e le misure sul testo attuale (motore/PROCEDURA.md §6; motore/editoriale/ruoli/00-diagnosta.md; motore/script/percorso.py);

4. `zb esito <libro> N`: parole, stile, continuità, audit anti-IA, lunghezze quasi uguali, capsula precedente;
5. `zb capsula <libro> N`, l'agente capsula, poi `--verifica`;

6. salvataggio con `zb salva`.

Il comando `/scrivi-capitolo N` fa questo giro e si ferma dopo il salvataggio (motore/editoriale/comandi/scrivi-capitolo.md).

Se un controllo fallisce: **una sola** correzione mirata, poi di nuovo `zb esito`. Al secondo fallimento di fila sullo stesso capitolo si apre `controllo_fallito`: decidi tu, con `ok` (accetti il capitolo così) o `correggi: …` (motore/PROCEDURA.md §6).

Ogni `pausa_lettura` capitoli c'è la pausa di lettura. Scritte tutte le unità del piano, la fase passa a «struttura_manoscritto» (motore/script/fase.py, dopo_capitolo; motore/script/percorso.py, dopo_stesura).

Passi 8-11. Le revisioni

Ogni fase di revisione ha due tempi: prima i **rapporti** (proposte), poi l'**applicazione** con modifiche puntuali e `zb esito N` sui capitoli toccati; `zb pronto` passa alla fase dopo. Ogni rapporto

comincia con la riga data da `zb ruolo` e sta in un commit che non tocca il testo rivisto (motore/PROCEDURA.md §1b; motore/script/percorso.py).

Fase	Chi	Report
struttura_manoscritto	diagnosta, poi editor strutturale	`rapporto-manoscritto.md`, `piano-revisione-manoscritto.md`
line	line editor, per capitolo (fino a 3 in parallelo)	`06-diagnostica/line/NN.md`
copy	correttore di italiano, per capitolo	`06-diagnostica/copy/NN.md`
bozze	`zb compila`, `zb impagina`, poi correttore di bozze	`06-diagnostica/bozze.md`

(motore/PROCEDURA.md §1b; motore/editoriale/ruoli/04-07; motore/editoriale/agenti/)

Un revisore non può riscrivere per intero un capitolo che esiste: l'hook rifiuta la riscrittura, si corregge con modifiche puntuali (motore/script/hook_recinto.py). Il comando `/revisiona N` chiama line editor o correttore secondo la fase (motore/editoriale/comandi/revisiona.md).

Passo 12. La chiusura meccanica

L'ordine è fisso: continuita, ortografia, compila, impagina, pdf, pacchetto, kdp, pronto. `pacchetto` va prima di `kdp`: il controllo KDP legge la scheda del pacchetto (motore/PROCEDURA.md §7).

Chiusura 1: `zb continuita`

- **Cosa fa.** Controlla giorni della settimana, età dei personaggi, durate, cifre, nomi scritti in due modi, ordine delle date.

- **Cosa serve prima.** I capitoli e il file `cronologia.yaml` del libro.
- **Cosa produce.** `06-diagnostica/continuita.md`, con la lista di controllo per capitolo.

(motore/script/continuita.py)

Chiusura 2: `zb ortografia`

- **Cosa fa.** Tre livelli: (a) Hunspell, sempre; (b) LanguageTool, solo se `ZB_LANGUAGETOOL` indica i suoi file; (c) una lista di controllo per la lettura umana, sempre, presa dalle sezioni 1-6 dei criteri del correttore.
- **Cosa serve prima.** Hunspell con il dizionario italiano; le parole ammesse del libro (`nomi_propri.txt`).
- **Cosa produce.** `06-diagnostica/controllo-ortografico.md`. Dice quali livelli sono stati eseguiti.

(motore/script/ortografia.py)

Chiusura 3: `zb compila`

- **Cosa fa.** Mette insieme il libro: frontespizio, copyright, sommario, parti, capitoli in ordine, appendici. Il sommario lo genera sempre il motore.
- **Cosa serve prima.** `libro.yaml` e i capitoli; le appendici dichiarate.
- **Cosa produce.** `05-output/<titolo>-completo.md`.

(motore/script/compila.py)

Chiusura 4: `zb impagina`

- **Cosa fa.** Il PDF di stampa con Chromium: margini dalla tabella KDP, pagine iniziali senza numero, sommario con le pagine vere, formato esatto della pagina, metadati Title e Author.
- **Cosa serve prima.** Il file compilato; Chromium e PyMuPDF.
- **Cosa produce.** `05-output/<titolo>.pdf`.

(motore/script/impagina.py)

Chiusura 5: `zb pdf`

- **Cosa fa.** Verifica il PDF: pagine, Title e Author uguali a `libro.yaml`, font incorporati, margini contro i minimi KDP, voci del sommario sulla pagina giusta.
- **Cosa serve prima.** Il PDF; gli strumenti `pdfinfo`, `pdffonts`, `pdftotext`.
- **Cosa produce.** `06-diagnostica/verifica-pdf.md`.

(motore/script/verifica_pdf.py)

Chiusura 6: `zb pacchetto`

- **Cosa fa.** Prepara la cartella per KDP con modelli compilati dai dati di `libro.yaml`. **Non** scrive testi commerciali (descrizione, quarta, parole chiave, categorie). Non sovrascrive mai un file che c'è già.
- **Cosa produce.** In `06-pubblicazione/`: `scheda-amazon.md`, `quarta.md`, `brief-copertina.md`, `conferme-autore.yaml`, `promemoria-ai.md`, `checklist.md`.

(motore/script/pacchetto.py)

Chiusura 7: `zb kdp`

- **Cosa fa.** Controlla la conformità a KDP in 15 sezioni. Ciò che deve confermare l'autore, finché non è confermato, risulta KO.
- **Cosa serve prima.** `libro.yaml`, la scheda del pacchetto, il compilato, il PDF, `conferme-autore.yaml`, i dati KDP del motore.
- **Cosa produce.** `06-pubblicazione/conformita-kdp.md`, una riga OK/KO per sezione.

(motore/script/conformita_kdp.py)

Chiusura 8: `zb pronto`

- **Cosa fa.** Controlla che ci siano tutti i risultati dei passi 1-7. Se sì, il libro passa a «chiuso»; se no, elenca cosa manca e con quale comando farlo.

(motore/script/fase.py, elenco CHIUSURA; motore/PROCEDURA.md §7)

> **Prima di pubblicare.** I dati KDP del motore non sono mai stati riverificati dall'autore (`ultima_verifica: null`).

> La verifica si registra con `zb verifica KDP fatta AAAA-MM-GG` (motore/dati/kdp.yaml; motore/report-motore.md §11).

Libri con le fermate di un tempo

I libri del percorso breve, quelli nati prima e quelli che chiedono dei gate nel briefing si fermano ancora ai loro gate. Il gate si apre con `zb pronto` e mostra i documenti a blocchi di circa 120 righe. Le risposte (motore/PROCEDURA.md §5):

- **avanti**: il blocco dopo. Non approva nulla.
- **ok**: approva. Vale solo se hai visto tutti i blocchi.
- **ok, lotti da N**: approva e fissa quanti capitoli scrivere di fila.
- **correggi: …**: registra una correzione; la fermata resta aperta.
- **stato**: mostra lo stato, senza cambiare nulla.

Qualunque altra frase è una domanda: non cambia lo stato (motore/PROCEDURA.md §5).

La traduzione

`zb traduci <libro> --a en inizia` crea il libro di traduzione in `08-traduzione-<lingua>/`: guida → stesura → line → copy → fedeltà → chiusura. Qui le fermate restano: guida e glossario si approvano al gate «guida» prima del capitolo 1, e ci sono i gate `line`, `copy`, `fedelta` (motore/PROCEDURA.md §1b; motore/script/traduci.py). KDP chiede di dichiarare una traduzione fatta con l'IA; per un'edizione in vendita è consigliata la lettura finale di un madrelingua (motore/editoriale/lingue/traduzione.md).

Dopo ogni passo: il salvataggio

`zb salva <libro> "<messaggio>"`: controlla che nessun file stia fuori dal recinto, rigenera lo stato compatto, fa commit e push con ripiego; poi `git`

`status -sb` e `git log -1`. Se il push fallisce, ci si ferma e si riporta l'errore. Mai dire «fatto» se il lavoro non è su GitHub (motore/script/salva.py; motore/PROCEDURA.md §8; CLAUDE.md).

4. Le regole

Cosa dice CLAUDE.md

In parole semplici (CLAUDE.md):

1. **Regola madre.** Il motore resta pulito; il ramo `motore` non contiene mai un libro.
2. **Briefing nuovo:** prima azione `zb nuovo`, che apre una sola cartella del libro. Da lì si scrive solo in quella cartella.
3. **Recinto.** Hook, controllo prima del commit e `zb avvio` rifiutano i file fuori dal libro attivo. Solo se scrivi «lavoro sul motore», Claude Code lancia `zb lavoro motore`.
4. **Letture.** Per un libro: `motore/PROCEDURA.md` e il `LEGGIMI.md` del libro; se manca il percorso, Claude Code lo chiede. `docs/` non si legge durante il lavoro.
5. **Avvio.** Prima di scrivere: `zb avvio <libro> [--ruolo R]` e la riga «Letto». Se si ferma, Claude Code si ferma e

dice perché. Un ruolo per sessione: senza `--ruolo` vale quello della fase.

6. **Cosa legge il motore:** solo `motore/`, la cartella del libro e la banca-voci (sola lettura).
7. **Orchestrazione.** La sessione principale non scrive prosa: chiama l'agente che indica `zb agenti prossimo <libro>`, un subagente per capitolo. Chi scrive non rivede (`separazione_ruoli`).
8. **Report di sessione** ogni 5 scambi o quando scrivi «passaggio», nella cartella del libro, con sette sezioni fisse.
9. **Precedenza:** una sola scala, in `motore/editoriale/LEGGIMI.md`; ogni contrasto si segnala.
0. **Metodo:** proponi, l'autore approva, applica. Mai approvare da solo. **Modo automatico:** sul percorso editoriale nessuna approvazione tra le fasi; ci si ferma solo alla pausa di lettura (3-5 capitoli), a un controllo fallito e per il canone.
1. **Risposte dell'autore:** ok, avanti, correggi, stato; alla pausa, ok oppure

correggi con le note. **Ramo:** quello della sessione, letto da git.

2. **Salvataggio** a ogni passo con `zb salva`, poi `git status -sb` e `git log -1`. Resoconto con codice, file e «branch allineato a origin: sì/no».

Gli hook, uno per uno

Il recinto: blocca

- **Cosa guarda.** Ogni scrittura di file (Write, Edit, MultiEdit, NotebookEdit) e i comandi di shell (Bash): redirezioni, `tee`, `sed -i`, `perl -i`, `cp`, `mv`, `rm`, `touch`, `mkdir` e altri.
- **Regola.** Si può scrivere solo in `libri/<slug-attivo>/`. Il resto è rifiutato.
- **Prima di `git commit` e `git push`** controlla la cartella unica: un commit che tocca il libro e altro è bloccato.
- **Revisori.** Un revisore non può riscrivere per intero un capitolo esistente: solo modifiche puntuali.
- **Chi scrive.** Annota chi scrive in `.zb/scritture.log` del libro, per la

separazione dei ruoli.

- **Libro attivo.** Sta nella cartella interna di git (`zb-libro-attivo`), non in un file salvato; i rami creati prima usano ancora `libri/libro-attivo.md`.
- **Lavoro sul motore.** Con `zb lavoro motore` si può scrivere nel repository, ma non in `libri/`.

(motore/script/hook_recinto.py; motore/PROCEDURA.md §10)

Limiti dichiarati. Su Bash il recinto non vede tutto: script che scrivono senza nominare il percorso, variabili o `cd` di comandi precedenti, `$(…)`, strumenti non elencati (`awk -i inplace`, `find -delete`, `xargs rm`). Li coprono il controllo di git prima di ogni commit, `zb avvio` e `zb salva` (motore/PROCEDURA.md §10).

Il manoscritto: avvisa

Se si scrive in `04-manoscritto/` senza `zb avvio` in quella sessione, stampa «ATTENZIONE: …», lo annota in `.zb/avvisi-hook.log` e lascia passare. La modalità sta in `motore/dati/hook.yaml`:

oggi `avviso`. Il passaggio a `blocco` è un commit a parte, dopo il tuo ok (motore/script/hook_manoscritto.py; motore/dati/hook.yaml).

La separazione dei ruoli: blocca

Chi ha scritto un testo non lo revisiona nella stessa sessione. In `motore/dati/hook.yaml` la scelta è `separazione_ruoli: blocco`: il motore **si ferma** al `zb pronto` delle fasi con rapporti e all'avvio di un ruolo di revisione. Puoi tornare ad `avviso` con un commit a parte. Una sessione che ha già un ruolo non ne prende un altro (motore/dati/hook.yaml; motore/script/ruolo.py).

La sessione e il consumo

L'hook di sessione scrive il codice della sessione; dopo una compattazione, i vecchi «avvio fatto» non valgono più (motore/script/hook_sessione.py). L'hook di consumo annota in `.zb/letture.log` i file letti e gli agenti chiamati; `zb consumo <libro>` ne fa il riassunto per sessione.

Misura righe e parole lette, non i token (motore/script/consumo.py).

Come si accendono

`zb hook attiva` mostra cosa scriverebbe; `zb hook attiva --ok`, solo dopo il tuo ok, crea `.claude/settings.json`. `zb hook stato` dice se sono accesi; `zb hook disattiva` li spegne (motore/PROCEDURA.md §10).

La cartella unica

Tutto ciò che riguarda un libro vive in `libri/<slug>/`. `zb cartella <libro>` controlla su git, e lo fanno da soli anche `zb avvio`, `zb push` e l'hook prima di commit e push (motore/script/cartella_unica.py):

- file modificati fuori dal libro;
- commit «misti», che toccano il libro e altro;
- titolo del libro in file fuori dalla sua cartella.

Le eccezioni sono in `motore/dati/cartella-unica.yaml`: `CLAUDE.md`, `.gitignore`, `libri/libro-attivo.md`, `.claude/settings.json`, `.claude/agents/`, `.claude/commands/`, e i merge del motore. I commit misti già fatti li dichiari tu, con il motivo: la storia non si riscrive (motore/dati/cartella-unica.yaml).

`zb push` e le prove sul motore

`zb push` fa il push del ramo corrente; se fallisce riprova, poi tenta un ramo alternativo, poi un `git bundle`. Sul ramo di un libro controlla prima la cartella unica. Sul **lavoro del motore** vuole una `zb prove` intera e tutta OK sullo stato attuale: le prove lasciano un'impronta, e senza quell'impronta il push non parte (motore/script/push.py; motore/script/prove.py).

Al punto 9 dell'integrazione le prove erano 632, tutte OK; al punto 8b, 633; al punto 12, 637 (docs/DECISIONI_INTEGRAZIONE.md, D12, D15 e D16). Oggi sono 675, tutte OK,

nessuna saltata (motore/report-motore.md §13).

Altre protezioni, senza hook

Chi	Cosa fa
Codice del motore	Ogni comando scrive solo nella cartella del libro
`zb nuovo`	Rifiuta un libro sul ramo `motore`, dentro `motore/` o nella radice
Controllo di git prima del commit	Rifiuta file fuori dal recinto, chiunque li abbia scritti
`zb recinto`	Prova che i comandi non scrivano fuori dal libro
`zb separazione`	Prova che il motore, `docs/` e `CLAUDE.md` non contengano libri né nomi vietati

(motore/script/nuovo.py; motore/script/precommit.py;

motore/script/recinto.py;
motore/script/separazione.py)

La precedenza

Quando due regole si contraddicono, vince quella più in alto. Una sola scala (motore/editoriale/LEGGIMI.md):

1. manuale del libro e `libro.yaml`;
2. briefing e scheda della voce (se contrastano vince il briefing, e si segnala);
3. profilo di genere;
4. `motore/editoriale/` (ruoli);
5. procedura del motore.

Un'eccezione decisa da te: con `frase_media_voce: true` nel profilo (thriller), per la frase media vale la voce, non il manuale (motore/editoriale/LEGGIMI.md).

Il catalogo anti-IA e i criteri del correttore sono filtri: non stanno sopra il manuale. **Ogni contrasto si segnala** (motore/editoriale/LEGGIMI.md; CLAUDE.md).

5. I rami

Il ramo `motore`

- Solo il motore: codice, regole, mini-libri inventati, e `docs/` con i documenti di lettura.
- Mai un libro, mai un nome di libro, di ramo di libro o di autore reale, anche in `docs/` e in `CLAUDE.md` (lo controlla `zb separazione`).
- Qui hook, agenti e comandi sono spenti; `zb avvio` senza libro lo verifica.
- Qui `zb nuovo` e `zb lavoro libro` si rifiutano.

(motore/passaggio-di-consegne.md, «Regole da non violare»; motore/PROCEDURA.md §4 e §10; motore/script/hook_recinto.py)

I rami dei libri

- Un libro per ramo, nella cartella `libri/<slug>/`.
- Ogni ramo porta una copia di `motore/`. Il motore aggiornato arriva con un merge: è l'unica eccezione che il

controllo prima del commit ammette (motore/script/precommit.py).

- Con hook e agenti accesi ci sono anche `.claude/settings.json`, `.claude/agents/`, `.claude/commands/`.
- I libri vecchi hanno il loro `CLAUDE.md` per tua scelta e non si toccano (motore/passaggio-di-consegne.md, «Regole da non violare»; motore/dati/cartella-unica.yaml).

I rami d'archivio

Esistono rami d'archivio. Per esempio, il ramo `motore` ha copiato `pulizia.py` su un ramo d'archivio prima di toglierlo; in questo ramo integrato `zb pulizia` c'è ancora, con le sue prove. I nomi dei rami d'archivio non si scrivono qui: lo vieta `separazione.py` (docs/ANALISI_REPOSITORY.md, tabella degli script; docs/DECISIONI_INTEGRAZIONE.md, D12 e D13).

Il ramo dell'integrazione

`integrazione-editoriale` sta su `NGTAProtocol/ZeusBot`. Parte dal ramo di sessione `ccr-d1e12273-izx78k` (`d2b8814`) e riprende dal ramo `motore` (`971b6df`) ciò che mancava. I rami `motore` e `main` non sono stati toccati; in questo repository un ramo `main` non esiste (docs/DECISIONI_INTEGRAZIONE.md, introduzione, D1 e D2).

6. Cosa c'è e cosa non esiste

Cosa c'è

- **Subagenti e comandi** di Claude Code: 12 agenti e 4 comandi, accesi sui rami dei libri con `zb agenti attiva --ok` (capitolo 2).
- **Anti-IA.** `zb esito` e `zb antiia` misurano e non riscrivono: variazione delle frasi, aperture uguali, formule per mille parole, lineette nella narrazione, varietà lessicale, chiusure uguali; in più le formule di `motore/editoriale/anti-ia/formule.yaml`, la coerenza tipografica e le ripetizioni ravvicinate. Gravità P0, P1, P2. Le soglie stanno in `motore/dati/antiia.yaml`, poi nel profilo di genere, poi in `libro.yaml`. Con `anti_ia: {modalita: blocco}` ogni segnalazione diventa un KO (motore/script/antiia.py; motore/dati/antiia.yaml).
- **Correttore di italiano**: il ruolo 6 e i criteri in `motore/editoriale/correttore/criter`

`i-correttore.md`, insieme a `zb ortografia` (motore/editoriale/ruoli/06-correttore-italiano.md).

- **LanguageTool**: un solo comando, `zb languagetool installa | verifica | stato`. Si installa fuori dal repository, da Maven Central, mai da languagetool.org (motore/script/languagetool.py).
- **Inglese**: «Lingua: en» nel briefing; stesse fasi e stessi ruoli, con i profili di `motore/editoriale/lingue/en/` (motore/PROCEDURA.md §1b).
- **Comandi di controllo singoli**: `zb conta`, `zb stile`, `zb riciclo`, `zb capitolo`, `zb pulizia` restano, oltre a `zb esito` (motore/dati/comandi.yaml; docs/DECISIONI_INTEGRAZIONE.md, D12).

Cosa non esiste

- **Skill di Claude Code** (`.claude/skills/`): non si usano. Agenti e comandi vanno in `.claude/agents/` e `.claude/commands/`; il livello editoriale sostituisce ogni skill

citata ma assente (motore/script/agenti.py; motore/editoriale/LEGGIMI.md, «Regole», punto 1).

- **Vocabolario anti-IA italiano**: i livelli L1-L3 sono vuoti finché non si estraggono dai dati con `zb antiia estrai` (motore/dati/antiia.yaml).
- **Nomi vietati**: `motore/dati/nomi_vietati.txt` non ha voci. Titoli, personaggi e luoghi dei libri veri li aggiungi tu; da solo `zb separazione` ricava, senza scriverli, i nomi dei rami di libro su origin e gli slug della banca-voci (motore/dati/nomi_vietati.txt; motore/report-motore.md §11, punto 10).
- **Un controllo automatico a ogni modifica** sul repository: non c'è. Le prove si lanciano con `zb prove`; sul lavoro del motore le lancia `zb salva` prima del commit (nessuna cartella .github/; motore/script/salva.py).
- **Un ambiente che resti** da una sessione all'altra: in un container nuovo

si reinstallano i requisiti (motore/report-motore.md §11, punto 9).

7. Appendici e indice

> Il motore **include** le appendici dal commit `71a36cc` («Motore: appendici dichiarate in libro.yaml e sommario completo», 14 ottobre 2026). Le regole sono in `motore/PROCEDURA.md` §7 (storia git; motore/script/compila.py).

Appendici. Sono i materiali dopo l'ultima unità del libro, epilogo compreso: soluzioni di enigmi, mappe, glossari, note (motore/PROCEDURA.md §7).

- Stanno nella cartella `05-appendici/` del libro (motore/dati/struttura-libro.yaml).
- Si dichiarano in `libro.yaml`, campo `appendici`, nell'ordine di lettura: titolo, file, didascalia per le immagini.
- Formati: testo `.md`; immagini `.svg`, `.png`, `.jpg`. Un `.svg` diventa PNG con Chromium.
- Ogni appendice comincia su una pagina nuova.
- Un file dichiarato e assente ferma `zb compila`. Un file presente ma non

dichiarato dà solo un avviso e resta fuori.
- Le appendici non vanno mai in `04-manoscritto/`.
- Un'appendice non anticipa nulla: le soluzioni stanno solo in appendice.
- Un libro senza il campo `appendici` si compila come prima.

(motore/PROCEDURA.md §7; motore/script/compila.py)

Indice (sommario). Lo genera `zb compila`, sempre. Non si scrive a mano.

- Contiene, in ordine: parti, prologo, capitoli («N. Titolo»), interludi, epilogo, appendici.
- `zb impagina` aggiunge la pagina vera di ogni voce; `zb pdf` controlla che sia giusta.
- Per cambiare una voce si cambia la sua fonte (titolo del capitolo, `libro.yaml`), poi si rilancia `zb compila`.

(motore/PROCEDURA.md §7)

8. Punti deboli e decisioni aperte

Sono i limiti noti di `motore/report-motore.md` §11, le scelte in attesa di `docs/DA_VERIFICARE_AUTORE.md` e di `docs/DA_FORNIRE.md`. L'ordine, dal più serio al più lieve, è una valutazione di questa guida.

Importanti: prima di pubblicare o fidarsi

1. **Dati KDP mai verificati da te.** In `motore/dati/kdp.yaml` c'è `ultima_verifica: null`. Vanno controllati sulle pagine ufficiali e registrati con `zb verifica KDP fatta AAAA-MM-GG`.
2. **Soglie anti-IA provvisorie.** Sono tarate sui mini-libri di prova, non su prosa umana, e nei profili sono una PROPOSTA uguale per tutti i generi (motore/dati/antiia.yaml; motore/profili/).
3. **Gli hook non vedono tutto** su Bash (capitolo 4). Restano il controllo di git,

`zb avvio`, `zb salva` e `zb push`.

4. **La separazione dei ruoli distingue le sessioni solo con l'hook di sessione** (o con `ZB_SESSIONE`). Con la sessione «locale» dà solo un avviso (motore/script/ruolo.py).
5. **Il lavoro si perde se non è su origin.** L'ambiente non è persistente: in un container nuovo vanno reinstallati PyMuPDF, markdown, Hunspell e LanguageTool.

Medi: misure e controlli incompleti

6. **Il consumo misura le letture, non i token.** Le prove controllano che con le capsule si leggano meno parole; le cifre misurate sui mini-libri dei due rami (1.127 parole con le capsule e 1.694 senza; 796 con e 2.444 senza) non sono state rifatte nell'integrazione (motore/report-motore.md §11, punto 6).
7. **La continuità ha limiti.** I tempi relativi si controllano solo se nessuna data nota coincide; le differenze d'età

solo per i personaggi con data di nascita completa.

8. **PDF: TrimBox e BleedBox non sono scritti;** trasparenze e immagini non sono controllate.
9. **I mini-libri sotto le 24 pagine** hanno la sezione 11 della conformità KDP in KO per costruzione.

Scelte tue ancora aperte

10. **Il pacchetto editoriale descrive ancora le fermate con l'ok** (`00-percorso.md`, la fase struttura, «solo dopo il mio ok» nei ruoli), mentre il motore scrive in automatico. `CLAUDE.md` è già allineato (D17). Se vuoi, aggiorna quei file (docs/DA_VERIFICARE_AUTORE.md).
1. **Sezione B del catalogo anti-IA.** Le sue formule stanno in `formule.yaml` ma non si contano finché `formule_sezione_b: false` (motore/dati/antiia.yaml; docs/DA_VERIFICARE_AUTORE.md).
2. **Coppia «avvenire/avvenire»** nei criteri del correttore, sezione 5: forse

un refuso, non toccata.

3. **Licenza di APODICTIC** (CC BY-NC-SA 4.0): nessun testo copiato.
4. **Corpus per il vocabolario anti-IA italiano**: servono prosa scritta da un modello e prosa umana dello stesso genere (docs/DA_FORNIRE.md).
5. **Separazione dei ruoli in blocco** e **capsule** con avviso (o `obbligatorie` in `libro.yaml`): sono scelte fatte per te; puoi cambiarle (motore/dati/hook.yaml; motore/script/capsula.py).
6. **File aggiunti dal ramo `motore`**, adattati ai nomi del pacchetto: vanno letti prima di considerarli tuoi (docs/DA_VERIFICARE_AUTORE.md).

Lievi

7. **La storia git contiene ancora i riferimenti tolti** e i documenti spostati sui rami d'archivio: non si riscrive (motore/report-motore.md §11, punto 13).

(motore/report-motore.md §11; docs/DA_VERIFICARE_AUTORE.md;

docs/DA_FORNIRE.md)

9. Dove trovare le cose

Ramo «motore» vuol dire anche: nella copia del motore di ogni ramo di libro aggiornato.

Cosa cerchi	File	Ramo
Regole di avvio	`CLAUDE.md`	motore (ogni ramo di libro ha il suo)
Procedura completa	`motore/PROCEDURA.md`	motore
Percorso e fermate	`motore/editoriale/00-percorso.md`	motore
Precedenza e regole editoriali	`motore/editoriale/LEGGIMI.md`	motore
Gli otto ruoli	`motore/editoriale/ruoli/`	motore
Diagnosi: principi e rapporto	`motore/editoriale/diagnosi/`	motore

Cosa cerchi	File	Ramo
Gli agenti	`motore/editoriale/agenti/`, `lingue/agenti/`	motore
Comandi di Claude Code	`motore/editoriale/comandi/`	motore
Modelli degli agenti	`motore/dati/modelli-ruoli.yaml`	motore
Orchestrazione e parallelo	`motore/dati/orchestrazione.yaml`	motore
Fase struttura e suoi modelli	`motore/editoriale/struttura/`	motore
Anti-IA	`motore/editoriale/anti-ia/`, `motore/dati/antiia.yaml`	motore
Criteri del correttore	`motore/editoriale/correttore/criteri-correttore.md`	motore

Cosa cerchi	File	Ramo
Glossari	`motore/editoriale/glossario.md`, `lingue/it/glossario-editoriale.md`	motore
Risparmio di token	`motore/editoriale/risparmio-token.md`	motore
Elenco dei comandi	`python3 motore/zb aiuto`; `motore/dati/comandi.yaml`	motore
Stato del lavoro sul motore	`motore/passaggio-di-consegne.md`	motore
Limiti noti del motore	`motore/report-motore.md` §11	motore
Scelte dell'integrazione	`docs/DECISIONI_INTEGRAZIONE.md`	integrazione

Cosa cerchi	File	Ramo
Da verificare e da fornire	`docs/DA_VERIFICARE_AUTORE.md`, `docs/DA_FORNIRE.md`	motore
Analisi dei repository esterni	`docs/ANALISI_REPOSITORY.md`, `docs/ATTRIBUZIONI.md`	motore
Progetto originale	`motore/proposta-motore.md`	motore
Profili di genere	`motore/profili/`	motore
Modelli vuoti	`motore/modelli/`	motore
Dati KDP	`motore/dati/kdp.yaml`	motore
Lista nera, parole ammesse	`motore/dati/lista-nera.yaml`, `parole-ammesse-it.txt`	motore
Cartelle di un libro	`motore/dati/struttura-libro.yaml`	motore

Cosa cerchi	File	Ramo
Campi di `libro.yaml` (anche `pausa_lettura`)	`motore/dati/libro.schema.yaml`	motore
Hook: script e modalità	`motore/script/hook_*.py`, `motore/dati/hook.yaml`	motore
Cartella unica: eccezioni	`motore/dati/cartella-unica.yaml`	motore
Output compatto	`motore/dati/output.yaml`	motore
Hook e agenti accesi	`.claude/settings.json`, `.claude/agents/`, `.claude/commands/`	ramo del libro
Impaginazione e font	`motore/stampa/`	motore

Cosa cerchi	File	Ramo
Prove e mini-libri inventati	`motore/script/prove.py`, `prove_editoriale.py`, `motore/prove/`	motore
Un libro	`libri/<slug>/`	ramo del libro
Briefing del libro	`libri/<slug>/00-progetto/briefing.md`	ramo del libro
Bibbia e codex	`libri/<slug>/02-bibbia/`	ramo del libro
Struttura del libro	`libri/<slug>/03-struttura/`	ramo del libro
Capitoli	`libri/<slug>/04-manoscritto/`	ramo del libro
Appendici	`libri/<slug>/05-appendici/`	ramo del libro
PDF e libro compilato	`libri/<slug>/05-output/`	ramo del libro

Cosa cerchi	File	Ramo
Report dei controlli e dei ruoli	`libri/<slug>/06-diagnostica/`	ramo del libro
Le tue note di lettura	`libri/<slug>/06-diagnostica/note-lettura.md`	ramo del libro
Pacchetto KDP	`libri/<slug>/06-pubblicazione/`	ramo del libro
Capsule	`libri/<slug>/07-capsule/`	ramo del libro
Report di sessione del libro	`libri/<slug>/passaggio-di-consegne.md`	ramo del libro
Voci narranti	`indice.yaml`, `voci/<slug>/`	banca-voci (repository a parte)
Questa guida	`docs/GUIDA_AL_SISTEMA.pdf` e `.md`	motore

(motore/editoriale/LEGGIMI.md; motore/dati/struttura-libro.yaml;

motore/PROCEDURA.md)

Fine della guida.